AthenaK Native Multigrid Tutorial

Concepts and code-reading guide

May 21, 2026

1 What Problem Multigrid Solves

Multigrid is an algorithm for solving elliptic equations, such as Poisson-like equations or the CTS constraint equations. These problems are difficult because local relaxation methods remove short-wavelength errors quickly, but long- wavelength errors decay slowly.

For a simple scalar problem,

$$Lu = f,$$

where L is a discretized differential operator, the solver tracks:

- u: the current solution estimate.
- f: the right-hand side or source.
- $r = f - L(u)$: the residual or defect.
- $e = u_{\text{exact}} - u$: the unknown error.

If u is wrong by a smooth, slowly varying error, pointwise smoothing on the fine grid sees only a tiny local slope and converges slowly. On a coarser grid, the same smooth error is represented by fewer cells and becomes easier to remove. The multigrid idea is therefore:

1. Smooth on the fine grid to remove oscillatory errors.
2. Restrict the remaining smooth residual/error information to a coarser grid.
3. Correct the coarse-grid problem.
4. Prolongate the coarse correction back to the fine grid.
5. Smooth again to remove interpolation noise.

2 A Minimal V-Cycle

The classic V-cycle looks like this:

```
V(level):
  if level is coarsest:
    solve or smooth many times
  else:
    pre-smooth u_level
    residual_level = f_level - L_level(u_level)
    restrict residual and solution to level-1
    V(level-1)
    prolongate coarse correction to level
    post-smooth u_level
```

The "V" name comes from the path through the levels:

fine	→	medium	→	coarse	→	medium	→	fine
smooth		restrict		solve		prolong		smooth

3 Linear Correction vs FAS

For a linear problem $Lu = f$, one often solves the error equation

$$Le = r.$$

For nonlinear systems, such as the CTS residual $F(u) = 0$, AthenaK uses the full approximation storage (FAS) pattern. In FAS, coarse levels store a full approximation u_H, not just an error correction. The coarse source is adjusted so that the coarse problem is consistent with the fine-grid solution:

$$F_H(u_H) = f_H.$$

In the current code, the physics-specific class supplies:

- a residual/operator evaluation,
- a smoother,
- a defect computation,
- an FAS right-hand-side update.

The generic multigrid driver supplies hierarchy traversal, restriction, prolongation, boundary exchange, root-grid handling, and convergence norms.

4 The Main Code Objects

File/class	Purpose
`src/multigrid/multigrid.hpp` `Multigrid`	Owns per-level arrays on MeshBlocks or on the root grid. Provides generic operations such as loading data, restricting, prolongating, computing norms, and storing old solutions.
`src/multigrid/multigrid.hpp` `MGOctet`	Small 2^3-interior cell object used to bridge static mesh-refinement coarse/fine boundaries. It stores `u`, `src`, `def`, `uold`, and optional coefficients.
`src/multigrid/multigrid_driver.cpp` `MultigridDriver`	Controls the global solve: builds the hierarchy, chooses current levels, runs V-cycles/FMG cycles, moves data between MeshBlocks/root/octets, and computes global norms with MPI reduction.
`src/multigrid/multigrid_tasks.cpp`	Builds task lists for boundary exchange, smoothing, restriction, prolongation, fine/coarse ghost filling, and send/receive cleanup.
`src/z4c/id_solve.hpp` `IDCTSMultigrid`	CTS-specific `Multigrid` subclass. Implements `SmoothPack`, `CalculateDefectPack`, and `CalculateFASRHSPack`.

File/class	Purpose
`src/z4c/id_solve.cpp` `IDCTSMultigridDriver`	CTS-specific driver. Sets solver parameters, coefficient count, stencil choices, loads CTS free data, calls generic multigrid cycles, and applies the solution.

5 The Arrays on Each Level

Each `Multigrid` object allocates one array per multigrid level:

Array	Meaning	CTS example
`u_`	current solution approximation	$\delta\Psi, \delta\beta^i$
`src_`	source/RHS for the level	zero on finest CTS level; FAS RHS on coarser levels
`def_`	defect/residual storage	$src - F(u)$
`coeff_`	problem coefficients/free data	conformal metric, K, α, sources
`uold_`	saved solution before coarse solve	used for FAS correction

The constructor `Multigrid::Multigrid()` creates either:

- MeshBlock-local levels when given a `MeshBlockPack`, or
- root-grid levels when given `nullptr`.

MeshBlock levels require cubic, power-of-two MeshBlocks. Root levels come from the number of root MeshBlocks in each direction. Each coarser level halves the number of active cells per direction.

6 Levels in AthenaK

There are three kinds of levels in this implementation:

6.1 MeshBlock Levels

These are the normal levels inside each MeshBlock. For a 64^3 MeshBlock, the internal multigrid levels are $64^3, 32^3, 16^3, \ldots, 1^3$, plus ghost zones. `mglevels_` owns these arrays.

6.2 Root Grid Levels

The root grid is built from the logical arrangement of root MeshBlocks. For a $2 \times 2 \times 2$ root MeshBlock layout, the root-grid active region starts as 2^3, then can coarsen to 1^3. `mgroot_` owns this part.

6.3 Octets for Mesh Refinement

For static mesh refinement, an octet represents a parent cell with $2 \times 2 \times 2$ refined children. Octets let the V-cycle pass through refinement levels without requiring every part of the domain to be uniformly refined.

For the native CTS solver, there are explicit guards:

- adaptive AMR is not supported,
- sixth-order CTS on SMR is not supported by the current octet bridge.

7 Setup Before a Solve

The generic setup path is in `MultigridDriver::PrepareForAMR()` and `MultigridDriver::SetupMultigrid()`.

7.1 `PrepareForAMR()`

Despite the name, this runs for every multigrid solve. It:

- records the root mesh level,
- detects whether the MeshBlock layout changed,
- recomputes how many refinement levels exist,
- reallocates MeshBlock multigrid arrays if needed,
- rebuilds octets if SMR/AMR levels exist,
- updates per-block cell widths.

7.2 `SetupMultigrid()`

This computes:

$$n_{\text{total}} = n_{\text{root}} + n_{\text{MeshBlock}} + n_{\text{refinement}} - 1.$$

It sets `current_level_` to the finest level and initializes the FMG level bookkeeping.

For CTS, `IDCTSMultigridDriver::Solve()` wraps setup with the physics data:

1. build CTS free data,
2. load `u_cts` into `u_`,
3. load zero source,
4. load free-data coefficients and fill their finest-level halos,
5. restrict coefficients and fill coefficient halos on each coarse level,
6. setup the hierarchy,
7. transfer coefficients to root and octets.

8 Restriction

Restriction moves information from a fine grid to a coarser grid. In a V-cycle, restriction is used on the way down.

`Multigrid::RestrictPack()` does:

1. call the physics-specific `CalculateDefectPack()`,
2. restrict `def_` to the coarser `src_`,
3. restrict `u_` to the coarser `u_`,
4. decrement `current_level_`.

For nonlinear FAS, restricting the current solution as well as the defect is important because the coarse grid stores a full approximation, not just an error.

In the octet path, `RestrictOctets()` computes each octet defect and writes the restricted defect/-source and restricted solution into either a coarser octet or the root grid.

9 Prolongation and Correction

Prolongation moves a coarse-grid correction back to a finer grid. AthenaK uses:

- `ProlongateAndCorrectPack()` for normal V-cycle corrections,
- `FMGProlongatePack()` for direct FMG initialization,
- `ProlongateAndCorrectOctets()` for octet coarse/fine paths.

The normal V-cycle path first computes a FAS correction:

$$\Delta u_H = u_H - u_{H,\mathrm{old}},$$

then prolongates Δu_H and adds it to the next-finer solution.

For MeshBlock-local levels, the code comments describe this as trilinear correction. For some root/octet bridge paths, the implementation uses a 3-by-3-by-3 coarse neighborhood with cubic-like weights.

10 Smoothing

Smoothing is the problem-specific local relaxation. The generic driver only knows it can call:

`pmg->SmoothPack(color).`

For CTS, `IDCTSMultigrid::SmoothPack()` dispatches to `SmoothImpl<2/3/4>()` based on the effective CTS stencil. The finest MeshBlock MG level uses the Z4c stencil, while coarser MeshBlock/root levels use `id_solve/mg_coarse_fd_stencil` (default `2` for higher-order fine runs). The smoothing kernel is red-black colored:

$$c = (\mathrm{color} + i + j + k) \bmod 2.$$

Cells of one color are updated first, boundary/ghost zones are refreshed, then the other color is updated. CTS now uses `id_solve/smoother_update=frozen_view`: before each smoothing visit the active level is copied into a dedicated frozen scratch view. Operator evaluations, diagonal updates, Newton-GS Jacobians, and line-search trial residuals read from the frozen view while accepted updates write to the live solution. This removes same-color read/write races for mixed and high-order stencils without expanding the task list to an 8/27/64-color schedule.

The default CTS update is:

$$u_v \leftarrow u_v - \omega \, \frac{F_v(u) - src_v}{D_v},$$

where D_v is an approximate diagonal of the CTS operator. This is an under-relaxed diagonal nonlinear smoother.

The CTS driver can also use `id_solve/smoother = newton_gs`. In that mode, each active red/black point forms the four-component local residual for $(\Psi, \beta^x, \beta^y, \beta^z)$, finite-differences a scale-aware local 4×4 Jacobian of the CTS operator, solves the coupled Newton correction, and applies a residual-checked damped update. If the local linear solve is singular, ill-conditioned, non-finite, or cannot produce a local residual decrease after backtracking, the code falls back to the diagonal update for that point. This coupled update is the better long-term algorithmic target for nonlinear FAS, but it is not automatically faster: it does more work at every active point. Production comparisons should therefore use defect reduction per wall-clock time, not stable ω alone.

Why red-black? A centered stencil couples nearest neighbors. If the grid is colored like a checkerboard, red cells depend mostly on black neighbors and vice versa. Updating one color at a time gives Gauss-Seidel-like in-place relaxation while preserving parallelism within each color. For CTS high-order and mixed derivative operators, the frozen view is what makes this two-color schedule race-free.

11 The Current V-Cycle in Code

The V-cycle entry point is:

`MultigridDriver::SolveVCycle(pdriver, npresmooth, npostsmooth)`.

It does:

```
startlevel = current_level
coffset_ ^= 1
while current_level > 0:
  OneStepToCoarser(npresmooth)
SolveCoarsestGrid()
while current_level < startlevel:
  OneStepToFiner(npostsmooth)
```

11.1 One Step to Coarser

`OneStepToCoarser()` depends on which kind of level is active:

- MeshBlock level: run the `mg_to_coarser` task list.
- Octet level: smooth octets, refresh octet boundaries, restrict octets.
- Root level: apply root boundary, optionally compute FAS RHS, smooth red/black with root boundary after each color, restrict root grid.

The MeshBlock task list includes:

1. start receive,
2. send boundary data,
3. receive/unpack boundary data,
4. physical boundary fill,
5. fine/coarse ghost fill,
6. FAS RHS computation,
7. red smoothing,
8. boundary exchange/fill,
9. black smoothing,
10. boundary exchange/fill,
11. optional extra red/black pair if `nsmooth > 1`,
12. restriction,
13. clear send/recv.

11.2 Coarsest Grid Solve

`SolveCoarsestGrid()` does not use a direct matrix solve. It applies root boundaries, stores old data, computes FAS RHS, then performs a number of red-black smoothing passes. The number of passes is roughly the size of the coarsest root grid in one dimension. If the coarsest root grid is 1^3 and average subtraction is enabled, it may zero-clear instead.

11.3 One Step to Finer

The upward path calls `OneStepToFiner()`. It prolongates a coarse correction and then performs post-smoothing. On MeshBlock levels, the `mg_to_finer` task list performs any needed boundary communication before prolongation, then boundary communication and red-black post-smoothing after prolongation. On the final step to the finest level, it performs a final boundary exchange so the defect norm sees up-to-date ghost zones.

12 Full Multigrid (FMG)

FMG is an optional startup strategy controlled by `full_multigrid`. Instead of beginning on the finest grid with a rough guess, FMG:

1. restricts the problem down to very coarse levels,
2. solves/coarsely smooths there,
3. prolongates to a finer level,
4. performs one or more V-cycles,
5. repeats until the finest level is reached,
6. then continues with the normal MG solve.

In code, this is `SolveFMG()`, `SolveFMGCoarser()`, and `FMGProlongate()`. CTS has `id_solve/full_multigrid` and `id_solve/fmg_ncycle` controls.

13 Boundary Handling

Boundary handling is essential because each smoother and residual evaluation uses ghost cells.

13.1 MeshBlock Boundary Exchange

MeshBlock task lists use:

- `StartReceive()` → `InitRecvMG()`,
- `SendBoundary()` → `PackAndSendMG()`,
- `RecvBoundary()` → `RecvAndUnpackMG()`,
- `ClearSend()` and `ClearRecv()` for cleanup.

These are scheduled around smoothing, restriction, and prolongation so stencil operations see valid ghost data.

CTS also exchanges and fills coefficient/free-data halos. After `LoadCoefficients()` and after each `RestrictCoefficients()` step, the coefficient arrays use the same same-level MG exchange buffers as `u_`. Their physical boundaries are filled as free data: periodic faces wrap and non-periodic faces use zero-gradient copies. When refinement is present, the coefficient arrays then call `FillFineCoarseMGGhosts()` so CTS residuals near fine/coarse interfaces do not read stale free-data ghosts. They do not use correction-variable odd reflection from `mg_zerofixed`.

13.2 Physical Boundaries

The base multigrid driver initializes multigrid physical BCs from mesh BCs:

- periodic mesh faces stay periodic,
- non-periodic mesh faces become `mg_zerofixed` in the generic base driver.

`mg_zerofixed` uses odd reflection of the correction variable. This approximately enforces zero value at the physical boundary. `mg_zerograd` uses even reflection. Multipole BC support exists in the base MG machinery and is used by gravity. The CTS driver overrides the non-periodic default with `id_solve/mg_bc`; its current default is `zerograd`, with `zerofixed` available for older behavior.

13.3 Fine/Coarse Ghosts

When SMR/AMR is present, `FillFCBoundary()` calls `FillFineCoarseMGGhosts()`. For root/octet paths, octet boundary helpers copy same-level data or interpolate from coarser neighbors. CTS also transfers coefficient/free-data arrays through this hierarchy.

The current octet coefficient bridge still uses the generic compact coarse-buffer path, not the full Z4c high-order prolongation kernels. That is why sixth-order CTS on SMR remains guarded off even though unigrid high-order CTS operators are supported. Octet CTS operators therefore default to `id_solve/mg_coarse_fd_stencil`; `id_solve/octet_fd_stencil` is an explicit override.

14 Residual and Norms

`CalculateDefectPack()` is physics-specific. For CTS it evaluates

$$def_v = src_v - F_v(u).$$

`Multigrid::CalculateDefectNorm()` then reduces this defect over active cells. For L1 and L2 norms the code multiplies by cell volume. The driver `MultigridDriver::CalculateDefectNorm()` performs an MPI all-reduce and divides by the total domain volume; L2 norms are square-rooted:

$$\|def\|_2 = \left(\frac{1}{V}\int |def|^2\, dV\right)^{1/2}.$$

For multi-variable systems like CTS, the CTS driver computes each variable norm and then reports the total as the Euclidean combination:

$$\|def\|_{\text{total}} = \sqrt{\sum_v \|def_v\|_2^2}.$$

15 How CTS Uses the Generic Multigrid

The CTS solve is a good concrete example:

1. `IDConformalThinSandwich::BuildFreeData()` builds all coefficients.
2. `IDCTSMultigridDriver::Solve()` loads the initial guess, source, and coefficients into the multigrid arrays.
3. Generic V-cycle machinery calls back into: `SmoothPack()`, `CalculateDefectPack()`, and `CalculateFASRHSPac`
4. `SmoothPack()` calls the selected CTS smoother (`diagonal` by default, or `newton_gs`).
5. `CalculateDefectPack()` evaluates the CTS residual.
6. `CalculateFASRHSPack()` adds the CTS operator value to the coarse FAS source.
7. After convergence or fixed V-cycles, `RetrieveResult()` copies the finest `u` back to `u_cts`.
8. `ApplySolution()` reconstructs ADM/Z4c data.

16 Fixed-Iteration vs Threshold Mode

The generic driver and CTS driver support two normal modes:

- **Threshold mode**: if `threshold >= 0`, V-cycles continue until the defect drops below the threshold or the iteration cap is reached.
- **Fixed-count mode**: if `threshold < 0`, the solver runs `niteration` V-cycles.

CTS adds best-solution guards:

- `keep_best_solution`: stores the best-so-far correction if later V-cycles diverge.
- `stop_on_defect_increase`: optionally stops when the defect grows beyond a tolerance.
- `reject_worse`: rejects the solved update if the final defect is worse than the initial defect.

17 How to Read the Multigrid Code

A useful reading order is:

1. `src/multigrid/multigrid.hpp`: learn the arrays, levels, `MGOctet`, and virtual physics hooks.
2. `src/multigrid/multigrid.cpp`: read `LoadFinestData`, `LoadSource`, `LoadCoefficients`, `RestrictPack`, `ProlongateAndCorrectPack`, and `CalculateDefectNorm`.
3. `src/multigrid/multigrid_driver.cpp`: read `SetupMultigrid`, `SolveVCycle`, `SolveCoarsestGrid`, `SolveFMG`, and `MGRootBoundary`.
4. `src/multigrid/multigrid_tasks.cpp`: read task-list construction for `mg_to_coarser` and `mg_to_finer`.
5. `src/z4c/id_solve.cpp`: read how CTS plugs into the generic hooks.

18 Mental Model for Debugging

When multigrid misbehaves, classify the failure:

- **Smoother instability**: defect grows during V-cycles. Try smaller ω, compare `smoother=diagonal` with `smoother=newton_gs`, inspect the local Jacobian/diagonal approximation, or use the best-solution guard.
- **Boundary/communication bug**: MPI stalls or rank-dependent results. Inspect task dependencies around `StartReceive`, `SendBoundary`, `RecvBoundary`, `ClearSend`, and `ClearRecv`.
- **Coarse-grid inconsistency**: fine-grid residual improves briefly but correction from coarse levels hurts. Inspect restriction, prolongation, and coefficient restriction.
- **Stencil/halo mismatch**: high-order operators use invalid ghost data or silently reduce order. Check stencil radius, ghost count, and whether any precomputed free-data fields need their own ghost halo.
- **Bad free data or source scaling**: initial defect changes strongly with resolution or physical constraints do not improve after solving.

For CTS specifically, the diagonal smoother remains the conservative default and is simple and parallel. The optional `newton_gs` smoother includes the local coupling among Ψ and β^i, but still needs damping and is more expensive because it finite-differences and solves a 4×4 system at each active point. Current production safeguards include a residual-checked line search, scale-aware local linear algebra, finite-value guards, update limits, and MPI-reduced smoother counters. Compare smoothers by defect reduction per wall-clock time rather than by stable ω alone. In the current 64^3, second-order, 40-cycle critical-Kerr comparison, `newton_gs` reached a lower final defect than `diagonal` at $\omega = 0.05$, but was still slower per wall-clock time.

19 Glossary

Term	Meaning
Smoother	Local relaxation step that damps high-frequency error.
Residual/defect	Difference between desired equation and current operator value, $f - L(u)$ or $src - F(u)$.
Restriction	Fine-to-coarse transfer.
Prolongation	Coarse-to-fine transfer.
V-cycle	Down to coarse levels, coarse solve/smooth, back to fine levels.
FAS	Full Approximation Storage, nonlinear multigrid method where coarse grids store full solutions rather than only corrections.
Root grid	Coarse grid built from the logical root MeshBlock layout.
MeshBlock level	Multigrid hierarchy inside each MeshBlock.
Octet	2^3-interior-cell structure that bridges refinement levels.
Ghost cells	Extra boundary cells needed for stencils and inter-block exchange.